

EbookAVPlayer — Codebase Audit & Verdict

Prepared from the uploaded `ebookavplayer-main.zip`. Ground-truth review, not a memory recall.

🕒 **Re-synced against the July 13 snapshot (`ebookavplayer-main_2.zip`), diffed file-by-file against the original July 9 review.** Status deltas since the first audit:

- **P1-4 (expression under-triggering) —  RESOLVED.** The Expression Sensitivity Plan is now marked *fully implemented* (Phases 0, 1a–1f, 2, 3a–3d, 4) and the code backs it. This was the single highest experiential recommendation in the whole review; it's built. See the rewritten P1-4 below.
- **P0-2 (Player god component) — still open, and it grew** from 1,290 → **1,342 lines**. This further validates the Simple Mode spec's additive (non-refactor) approach.
- **P1-1 (orphaned `timing.test.mjs` / missing `isCheckpoint`) — still open** (5 references in the test, 0 definitions in `timing.js`).
- **P0-1 (dual backend) — still open and reinforced:** `server/analyze/prompt.py` and `schema.py` were edited again this cycle, i.e. expression-era changes are being maintained in *both* the Worker and the legacy Python server — precisely the drift cost this finding warns about.
- **Net-new since July 9 (all positive):** a fully-wired expression subsystem (`expression-bucket` / `-prosody` / `-repass` + two queue consumers + `audit_expression.py`), vision-based illustration→character matching (`illustration-character-match.js`, confirmed-only), a per-scene **tension** curve, a **Performance Mode** dial, a read-only **Director's Log**, MLX local extraction, and a much larger test suite. The verdict below is unchanged in shape but stronger: the "make it a performance" gap is now largely closed.

How much of this I actually read

I want to be straight about coverage so you can weight the findings correctly.

- **Read in full:** `README.md`, `HANDOFF.md`, `CURSOR_HANDOFF.md`, `SETUP.md`, `visual-audiobook-brief.md`, `docs/MANUAL_TEST_TODO.md`, `docs/FUNNY_IDEAS_BACKBURNER.md`, `docs/ART_STYLES.md` (+ 3 more docs substantially), `.env.example`, `worker/worker.js`, `worker/_shared/pipeline-registry.js`, `worker/_shared/pipeline-cost-guide.js`,

`web/src/App.jsx` , `web/src/audio/orchestrator.js` ,
`web/src/components/Controls.jsx` , `web/src/audio/voicePrefs.js` .

- **Mapped structurally (not line-by-line):** `web/src/components/Player.jsx` (1,290 lines), `PlayerMenu.jsx` , `Library.jsx` , `worker/_shared/freemium-extract.js` .
- **Surveyed:** full file tree, line counts, dependency manifests, and I ran the Node test suite in a sandbox (no network, no `npm install`).
- **Did NOT read line-by-line:** the remaining ~280 files (most of `web/src/components/*` , the Python `server/` tree, the `worker/queue/*` consumers, most tests). Findings about those are inferred from names, imports, and the modules that reference them — treat them as leads, not verified facts.

So: this is a thorough architectural review, not a line-by-line audit of 34k lines. Anywhere I'm inferring, I say so.

Verdict (the stamp you asked for)

This is strong, senior-level work. It earns the stamp — with an asterisk.

The engineering instincts on display are genuinely good, in ways that are rare:

1. **The timing orchestrator (`web/src/audio/orchestrator.js`) is the best file in the repo.** It does exactly what the brief demanded ("keep one central orchestrator") and it does it with real rigor: a clean split between *Mode A* (estimated per-line clock, used for TTS/silent playback) and *Mode B* (acoustic continuous clock that reads the real `<audio>` playhead and maps it back to a line via `lineAt()`), progressive WhisperX timeline extension *mid-playback* without a reset, buffering surfaced to the UI, and a `resyncDisplay()` that correctly handles the rAF-suspended-while-tab-hidden problem. Every non-obvious decision has a "why" comment written for the next agent. This is production-grade.
2. **The provider architecture is disciplined and honest about its own constraints.** `pipeline-registry.js` + `pipeline-cost-guide.js` encode a KV-persisted, reorderable fallback chain per lane (extract / image / image-freemium-character / image-freemium-background) and then *measure the live config against a cost-efficient preset and emit actionable tips*. The Ollama stage notes contain real empirical benchmarking (tok/s solo vs. 8-way concurrency, MoE active-param counts). The `$0` ceiling isn't a vibe — it's enforced structurally (`EXTRACT_SKIP_GEMINI` defaults true; Gemini Image disabled in the recommended preset).

3. **Dependency discipline is excellent.** Root `package.json` has **zero runtime dependencies** (only `wrangler`, `fflate`, `pngjs`, `jpeg-js` as devDeps). The web app is React + Clerk and almost nothing else. For a project this capable, that's remarkable restraint and it will age well.
4. **The async architecture is correct for the platform.** `docs/CLOUDFLARE_BACKEND.md` shows you understood the real failure mode (Workers CPU/wall-clock limits) and moved to return-immediately + Queue + R2 + KV polling. The edge worker's "fast-path handler, else fall through to `onRequest` origin proxy" pattern in `worker.js` is a clean migration seam.
5. **The docs are a genuine asset.** `EXPRESSION_SENSITIVITY_PLAN.md` in particular is better root-cause analysis than most professional teams produce — it quantifies the problem (246/3,022 lines tagged non-normal, but Chapter 1 came back 100% normal on *both* cloud and local models) and isolates four independent leaks. That's an engineer who debugs with evidence.

The asterisk: the project is carrying real structural debt that will slow every future change if not addressed — a dual-backend maintenance burden, a 1,290-line god component, doc/reality drift, and a test suite that is not fully green even with deps. None of these are fatal. All are the predictable cost of a fast-moving solo project. They're the subject of the rest of this document and of `02_REVOLUTION_ROADMAP.md`.

If a senior reviewer at a good shop opened this repo, their reaction would be "this person can architect" followed by "...and this needs a consolidation pass before it scales." Both are true.

Architecture at a glance

Three layers, ~34k lines of source + ~10k of tests:

web/	React + Vite client	~19.5k LoC	← heaviest; where the UX debt li
worker/	Cloudflare Workers (JS)	~10.7k LoC	← the CURRENT backend
server/	Python FastAPI	~8.3k LoC	← LEGACY / reference (per README

Data flow (current, edge path): Browser → `/projects/ebookavplayer/api/*` → `worker.js` → {edge fast-path: R2/KV/Queue} OR {fall through to `VAE_API_ORIGIN` proxy} . **Ingest** returns a `job_id` immediately; queue consumers (`worker/queue/*-consumer.js`) do extraction, imaging, pack build, moment generation, and re-extraction; the client polls `/ingest/:id` and streams job events via a Durable Object (`job-event-hub.js`).

The pipeline itself: **parse EPUB → chunked LLM extraction (chapter-checkpointed) → character reconcile/merge → image gen (freemium chain) → compile playback JSON → Edge TTS at play time**, with a four-tier audio↔script timing engine on the client (`web/src/timing/*`) topped by a local WhisperX forced-alignment tier.

Findings

Ordered by how much they'll hurt you over the next six months. Severity: **P0** = fix before building more; **P1** = fix soon; **P2** = hygiene.

P0-1 — Two backends, one product, drifting apart

`server/` (Python) and `worker/` (JS) implement overlapping pipelines. The README says `worker/` is current and `server/` is "reference only," but the evidence shows the split is leakier than that label implies:

- `docs/MANUAL_TEST_TODO.md` (dated 2026-06-19, *after* the Workers port) writes its P0/P1 fixes against **Python paths** (`server/analyze/prompt.py` , `server/playback/compile.py` , `server/images/generate.py`) — and marks them all done. So either those fixes were applied only to the legacy backend, or applied twice, or the doc is stale. All three possibilities are a problem.
- `worker.js` 's health check still reports `origin: Boolean(env.VAE_API_ORIGIN)` and every edge handler can fall through to the Python origin. So the Python server isn't purely reference — it may still be a live proxy target in some deployments.
- `pipeline-registry.js` 's header comment says it "mirrors `server/pipeline/registry.py` ." Two files that must stay in lockstep, by hand, is a bug factory.

Why it's P0: every feature now has an ambiguous home. "Where do I fix expression tagging?" has two answers, and the docs point at the wrong one. This is the single biggest tax on the codebase.

Recommendation: pick one. Given the deployment story (Cloudflare-first, \$0 , portfolio embed), **`worker/` is the winner**. Formally retire `server/` : move the one piece still genuinely used (`server/align/forced_aligner.py` , imported by `scripts/local-align-server`) into `scripts/` , delete or archive the rest to a `legacy/` branch, and strip the origin-proxy fall-through from `worker.js` once nothing needs it. Update every doc that references a `server/...` path. This is a day or two of work that pays back immediately. Full plan in `02_REVOLUTION_ROADMAP.md` §1.

P0-2 — `Player.jsx` is a 1,342-line god component (grew from 1,290 since July 9)

It owns, in one file: playback state, sleep timer, TTS-error modal state, M4B attach/detach + timeline computation, alignment event log, five different sheet open/close booleans, the illustration-flash state machine (7 pieces of state: `activeFlash`, `flashActive`, `flashManual`, `flashDismissSignal`, plus 3 refs), moment generation, fullscreen, chapter navigation, progress scrubbing, resume persistence, job-event subscription, and regen-watching. `~30 useState` + `~15 useEffect`.

Why it's P0: this is where you'll spend the UI-overhaul effort, and it's currently unsafe to change — too many interdependent effects, too much implicit coupling. It's also the highest-risk file for a smaller model (Haiku) to edit without breaking something three concerns away.

Recommendation: extract cohesive hooks *before* the UI overhaul, not during. Concretely: `useIllustrationFlash()`, `useM4bTimeline()`, `useSleepTimer()`, `usePlayerSheets()`, `usePlaybackResume()`. Each is a mechanical, test-guarded extraction (the e2e specs in `web/tests/e2e/` are your safety net). Target: `Player.jsx` under ~400 lines that reads as orchestration, not implementation. This is a prerequisite for `03_UI_OVERHAUL...` and is sequenced there.

P1-1 — The test suite is not green, and one failure is a real bug

Running the Node suite in a clean sandbox: **10/20 pass**. Nine failures are just missing `npm install (fflate, jpeg-js not present)` — not real bugs; they'll pass on your machine. **But `tests/timing.test.mjs` fails for a real reason:** it imports `isCheckpoint` from `web/src/audio/timing.js`, and that export **no longer exists** (current exports: `estimateDurationSec`, `revealedCount`, `effectiveLineDuration`, `lineGapMs`, `sleepTimerRemainingMs`, `slotXForIndex`, `stageLayout`). The checkpoint logic was refactored out and the test was orphaned.

Why it matters: a red suite trains you to ignore red. And it means "checkpoints" (a brief-level feature — the fall-asleep catch) may have lost its unit coverage entirely.

Recommendation: decide where checkpoint logic lives now (I suspect `lineKinds.js` or it moved into the component), update or delete `tests/timing.test.mjs`, and get the Node suite to 20/20 on a machine with deps. Add a single `npm test` root script that runs all `tests/*.test.mjs` so "is it green" is one command. (Today each test is its own npm script — there's no aggregate runner at the root, which is *why* an orphaned test went unnoticed.)

P1-2 — Documentation drift is actively misleading

Beyond the MANUAL_TEST_TODO issue: `HANDOFF.md` and `CURSOR_HANDOFF.md` describe a FastAPI-only MVP and are explicitly “point-in-time,” but they still contain a “Stub / seam registry” and “architecture guardrails” that a new agent (or Haiku) will read as current — including the guardrail “Edge TTS server-side — keep `/tts` on FastAPI,” which is now wrong (it’s on the Worker). `docs/ECOSYSTEM_INTEGRATION.md` still lists EbookAVPlayer’s stack as “FastAPI + React/Vite.”

Recommendation: add a one-line `> STATUS: HISTORICAL — predates Workers port` banner to the top of both handoffs (you have this on some but not consistently applied to the guardrail sections), and fix the one-liner in the ecosystem doc. Cheap, high-leverage: these docs are what you’ll feed future agents.

P1-3 — The “note” system is a single technical string

`App.jsx` surfaces all status/errors through one `note` string rendered in a floating div, with developer-facing language: “*Backend unreachable — embedded demo. Import an offline pack for real books.*” There’s a richer `BannerStack / banners.js` system too, but the top-level app messaging is the blunt version.

Why it’s P1 (and central to your accessibility goal): your mother will hit these messages and they mean nothing to her. This is addressed in detail in

`03_UI_OVERHAUL_SIMPLE_MODE_HAIKU_SPEC.md`, but it’s flagged here because it’s a correctness/UX issue independent of Simple Mode.

P1-4 — Expression under-triggering — RESOLVED (July 13)

Original finding (July 9): the expression field existed end-to-end but fired on ~8% of lines, with whole chapters coming back flat, due to four leaks — a buried one-line instruction (`dialogue-rules.js:116`), a flat 0.2 extraction temperature that collapsed subjective calls to “normal”, no audit/repair loop, and downstream discarding over half the signal. *Status now:* `docs/EXPRESSION_SENSITIVITY_PLAN.md` is marked **fully implemented (Phases 0, 1a–1f, 2, 3a–3d, 4)**, and the tree matches the claim:

- **Leaks 2 + 3 (temperature, no repair loop):** `worker/_shared/expression-repass.js` — a focused second pass over dialogue/thought lines only (small `{index, expression, intensity}` payload, no chapter text), run at temperature **0.55**, gated by `VAE_EXPRESSION_REPASS`, and auto-triggerable off a “suspiciously flat” threshold that mirrors `scripts/audit_expression.py`. That auto-trigger *is* the feedback loop leak 3 said was missing. Dispatch routes `expression-repass` as a real queue job.
- **Leak 4 (downstream discard):** `expression-bucket.js` (mirrored in `web/src/expressionBucket.js`) normalizes the 70+ freeform values the model still emits onto **16 canonical buckets**, so nothing falls through to an inert CSS class or a no-

op prosody offset; `expression-prosody.js` maps bucket + intensity → Edge-TTS pitch/rate/volume (additive on the per-character base from `voice-assign.js`); the `expression-sprites` consumer renders the visual half.

- **Beyond the original scope (Phase 4):** a `tension.js` curve that builds across consecutive high-intensity lines and decays on calm; a **Performance Mode** dial (subtle/balanced/full) that scales the prosody table; and a read-only **Director's Log** overlay (`DirectorsLog.jsx`) that surfaces the resolved expression/intensity/bucket per line for tuning. *Worth confirming (why this is "resolved pending a check," not blindly closed):* verify `VAE_EXPRESSION_REPASS` is actually enabled in your deployed config — it's the pass doing the heavy lifting, and it's off by default — and re-run `audit_expression.py` on a chapter that was previously flat to confirm the non-normal ratio genuinely moved. If both hold, this is closed.

P2-1 — `ReplaceArtSheet.jsx` (867 lines) is the next god-component after Player

Not urgent, but it's the second-largest component and almost certainly has the same "too many concerns" smell. Worth a look when you touch art replacement.

P2-2 — CSS is two flat files (~930 lines) with no layering system

`styles.css` (817) + `styles-components.css` (114). It's competent — proper CSS custom properties for theming, dark/light, sprite spotlight/dim states, expression filters — but it's a single flat namespace (`vae-*`). This is *fine* at current size and I would **not** rewrite it. But the Simple Mode work will add UI, so `03_...` specifies exactly how to add tokens without fighting the existing cascade (the skill's warning about `.section` vs `.cta` specificity collisions applies).

P2-3 — Smoke/probe artifacts are committed to the repo

`smoke_out/freemium_probe/` contains ~40 committed PNGs from live provider runs, plus `data/packs/cache/*.vaepack`. These bloat the repo and clones. Consider `.gitignore`-ing generated probe output.

What I explicitly did NOT find wrong (so you don't over-correct)

- **The CommonJS/pure-module style is not a problem.** It's a deliberate, coherent choice and it's executed consistently. Don't let anyone talk you into a framework.
- **The multi-provider fallback is not over-engineering.** For a \$0 ceiling with flaky free

tiers, redundancy *is* the product. Keep it.

- **Controls.jsx is already good.** The Spotify/Audible-style transport dock is clean, has `aria-label`s, and is close to accessible as-is. Simple Mode builds on it rather than replacing it.
 - **The orchestrator does not need refactoring.** Leave it alone. It's the one file that's earned the right to be complex.
-

Bottom line

You asked whether this warrants praise from readers and tech enthusiasts. Honestly: **the architecture and the orchestrator would**. The provider-chain design and the `$0 - enforced-structurally` story are a genuinely good conference-talk or blog-post narrative. What would stop a discerning reader short is the dual-backend ambiguity and the god components — not because they're unusual (they're the normal entropy of a solo project moving fast) but because they're exactly the things a strong engineer is expected to notice and pay down.

So the honest framing for a public showcase is: *"a capable, cost-obsessed, free-tier-first visual-audiobook engine with a rigorously-designed timing core — mid-consolidation."* Do the P0 consolidation and the Simple Mode overhaul, and it's not mid-anything. It's a portfolio centerpiece.

Next: `02_REVOLUTION_ROADMAP.md` (where to take it) and

`03_UI_OVERHAUL_SIMPLE_MODE_HAIKU_SPEC.md` (the pedantically-specific build spec).